

# Informe de Pruebas Unitarias, Integración y E2E

## Resumen ejecutivo

La solución implementa **17 pruebas totales** distribuidas en tres niveles:

- **4 Unit Tests** (BffServiceApplicationTests): Validación de controllers con mocks
- **6 Integration Tests** (AcademicoIntegrationTest): Validación de composición BFF ↔ microservicios
- **7 E2E Tests** (AcademicoE2ETest): Validación de flujos de negocio completos

**Estado:** 17/17 PASANDO (última ejecución: 2026-06-12)

## Herramientas utilizadas

- **Framework de testing:** JUnit 5 (Jupiter)
- **Mocking:** Mockito 5.x, Spring Boot Test MockMvc
- **Stub de servicios:** OkHttp MockWebServer (puerto 8081 en tests)
- **Aserción:** Hamcrest Matchers, AssertJ (opcional)
- **Configuración de tests:** Spring Boot @SpringBootTest, @WebMvcTest
- **Cobertura:** JaCoCo (configurable)

## Estructura de tests por componente

### 1. BffServiceApplicationTests - Unit Tests (4 tests)

**Ubicación:** bff-service/src/test/java/com/colegio/bffservice/ **Anotaciones:** @WebMvcTest(AcademicoController), @AutoConfigureMockMvc(addFilters=false) **Propósito:** Validar controllers de BFF con dependencias mockeadas

#### ✓ **Test 1: contextLoads()**

- **Objetivo:** Verificar carga de contexto Spring
- **Tipo:** Smoke test
- **Resultado:** PASS

#### ✓ **Test 2: alBuscarPorCurso\_debeRetornarListaDeEstudiantes()**

- **Endpoint:** GET /academico/curso/{curso}
- **Entrada:** curso = "1-A"
- **Mock:** academicoFacade.obtenerEstudiantesPorCurso() retorna lista de 2 estudiantes

- **Aserción:** `status().isOk(), jsonPath("$", hasSize(2))`, nombres en respuesta
- **Resultado:** PASS

#### ✓ **Test 3: *alCrearAsistencia\_debeRetornarObjetoCreado()***

- **Endpoint:** POST /academico/asistencias
- **Body:** `{"estudianteId": 1, "presente": true}`
- **Mock:** `academicoFacade.marcarAsistenciaEstudiante(1L, true)` retorna `{"status": "creado"}`
- **Aserción:** `status().isOk(), jsonPath("$.status", is("creado"))`
- **Verificación:** Mockito verifica 1 llamada al facade con argumentos correctos
- **Resultado:** PASS

#### ✓ **Test 4: *alCrearEvaluacion\_debeRetornarObjetoCreado()***

- **Endpoint:** POST /academico/evaluaciones
- **Body:** `{"estudianteId": 1, "materia": "Matemáticas", "nota": 6.5}`
- **Mock:** `academicoFacade.guardarEvaluacionEstudiante(1L, "Matemáticas", 6.5)` retorna `{"id": 100}`
- **Aserción:** `status().isOk(), jsonPath("$.id", is(100))`
- **Verificación:** Mockito verifica parámetros exactos
- **Resultado:** PASS

## 2. AcademicoIntegrationTest - Integration Tests (6 tests)

**Ubicación:** `bff-service/src/test/java/com/colegio/bffservice/` **Anotaciones:** `@SpringBootTest, @AutoConfigureMockMvc(addFilters=false)` **Propiedades de test:**

```
eureka.client.enabled=false
estudiante.service.url=http://localhost:8081
asistencia.service.url=http://localhost:8081
evaluacion.service.url=http://localhost:8081
```

**Propósito:** Validar comunicación BFF → MockWebServer stub services

#### ✓ **Test 1:**

***testObtenerDatosAcademicosPorId\_DebeComponerDatosDeMultiplesServicios()***

- **Objetivo:** Verificar que BFF compone datos de múltiples servicios
- **Flujo:** BFF llama a 3 endpoints del stub (estudiante + asistencias + evaluaciones)

- **Endpoint:** GET /academico/{estudianteId}
- **Aserción:** status().isOk(), jsonPath("\$.estudiante", notNullValue())
- **Resultado:** PASS

✓ **Test 2: testCrearAsistencia\_DebeGuardarEnAsistenciaService()**

- **Objetivo:** Guardar asistencia en stub de asistencia-service
- **Endpoint:** POST /academico/asistencias
- **Body:** {"estudianteId": 1, "presente": true}
- **Stub:** MockWebServer en :8081 responde con {"status": "creado"}
- **Aserción:** status().isOk()
- **Resultado:** PASS

✓ **Test 3: testCrearEvaluacion\_DebeGuardarEnEvaluacionService()**

- **Objetivo:** Guardar evaluación en stub
- **Endpoint:** POST /academico/evaluaciones
- **Body:** {"estudianteId": 1, "materia": "Matemáticas", "nota": 7.5}
- **Stub:** Responde con {"id": 100}
- **Aserción:** status().isOk()
- **Resultado:** PASS

✓ **Test 4: testObtenerEstudiantesPorCurso\_DebeRetornarLista()**

- **Objetivo:** Listar estudiantes de un curso
- **Endpoint:** GET /academico/curso/{curso}
- **Stub:** Retorna [{"id": 1, "nombre": "Juan Perez", "curso": "1-A"}]
- **Aserción:** status().isOk()
- **Resultado:** PASS

✓ **Test 5: testObtenerDatosAcademicosPorRun\_DebeRetornarDatos()**

- **Objetivo:** Buscar estudiante por RUN
- **Endpoint:** GET /academico/run/{run}
- **Stub:** Retorna estudiante o 404 (ambos válidos)
- **Aserción:** status == 200 || status == 404
- **Resultado:** PASS

✓ **Test 6:**

**testMarcarAsistenciaCurso\_DebeActualizarTodosLosEstudiantes()**

- **Objetivo:** Registrar asistencia para todo un curso
- **Endpoint:** POST /academico/curso/{curso}/asistencia
- **Body:** {"presente": true}
- **Stub:** Retorna lista de confirmaciones
- **Aserción:** status().isOk()
- **Resultado:** PASS

### **3. AcademicoE2ETest - End-to-End Tests (7 tests)**

**Ubicación:** bff-service/src/test/java/com/colegio/bffservice/ **Anotaciones:** Idem AcademicoIntegrationTest **Propósito:** Validar flujos de negocio completos (Frontend → BFF → Stubs)

✓ **Test 1: e2e\_CrearEstudianteYVerificarRegistro()**

- **Flujo:** Crear → Verificar registro → Obtener datos
- **Pasos:**
  1. POST /academico/estudiantes con {nombre, run, curso}
  2. GET /academico/run/{run} para verificar sync
- **Propósito:** Validar persistencia end-to-end
- **Resultado:** PASS

✓ **Test 2: e2e\_RegistrarAsistenciasCursoCompleto()**

- **Flujo:** Obtener estudiantes → Registrar asistencia grupal
- **Pasos:**
  1. GET /academico/curso/{curso} → Lista estudiantes
  2. POST /academico/curso/{curso}/asistencia con {presente: true}
- **Propósito:** Validar operación batch
- **Resultado:** PASS

✓ **Test 3: e2e\_CrearYCalificarEvaluacion()**

- **Flujo:** CRUD completo de evaluación
- **Pasos:**
  1. POST /academico/evaluaciones → Crear
  2. PUT /academico/evaluaciones/{id} → Actualizar nota

3. DELETE /academico/evaluaciones/{id} → Eliminar

- **Propósito:** Validar ciclo completo
- **Resultado:** PASS

✓ **Test 4: e2e\_EstudianteVerificaSuRendimiento()**

- **Flujo:** Obtener vista consolidada de rendimiento
- **Pasos:**
  1. GET /academico/{estudianteId} → Retorna AcademicoDTO consolidado
- **Propósito:** Validar composición de datos
- **Resultado:** PASS

✓ **Test 5: e2e\_OperacionesCRUDEstudiante()**

- **Flujo:** Operaciones CRUD en estudiantes
- **Pasos:**
  1. POST /academico/estudiantes → Crear
  2. GET /academico/{id} → Leer
  3. DELETE /academico/estudiantes/{id} → Eliminar
- **Propósito:** Validar CRUD base
- **Resultado:** PASS

✓ **Test 6: e2e\_RegistrarAsistenciaIndividual()**

- **Flujo:** Registrar asistencia → Verificar en datos académicos
- **Pasos:**
  1. POST /academico/asistencias → Registrar
  2. GET /academico/{id} → Verificar en array asistencias
- **Propósito:** Validar sincronización
- **Resultado:** PASS

✓ **Test 7: e2e\_RegistroLoginYAccesoADatos()**

- **Flujo:** Autenticación → Acceso a datos
- **Pasos:**
  1. POST /auth/register → Registrar usuario
  2. POST /auth/login → Generar JWT
  3. GET /academico/{id} con Authorization: Bearer <token> (si aplica)
- **Propósito:** Validar flujo de autenticación

- **Resultado:** PASS

## Cobertura por componente

| Componente                        | Tests                  | Estado | Cobertura Est. |
|-----------------------------------|------------------------|--------|----------------|
| bff-service / AcademicoController | 17                     | PASS   | 70%+           |
| bff-service / AcademicoFacade     | 6 (Integration)        | PASS   | 60%+           |
| MockWebServer stubs               | 13 (Integration + E2E) | PASS   | 80%+           |

## Instrucciones para ejecutar pruebas y generar reportes

### Ejecutar todos los tests del BFF

```
cd bff-service
.\mvnw test
```

### Ejecutar tests específicos

```
# Solo unit tests
.\mvnw test -Dtest=BffServiceApplicationTests

# Solo integration tests
.\mvnw test -Dtest=AcademicoIntegrationTest

# Solo E2E tests
.\mvnw test -Dtest=AcademicoE2ETest

# Todos los tests principales
.\mvnw test "-
Dtest=AcademicoIntegrationTest,AcademicoE2ETest,BffServiceApplicationTests"
```

### Generar reporte JaCoCo (cobertura)

```
# Build + test + cobertura
.\mvnw clean test jacoco:report

# Abrir reporte HTML
start target/site/jacoco/index.html
```

## Interpretación de resultados

### Resultado esperado

```
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

### Acciones recomendadas

- Si hay fallos: Revisar MockWebServer stubs en

AbstractMockWebServerTest.java

- Si Eureka falla: Verificar que `eureka.client.enabled=false` en `application-test.properties`
- Si conexión localhost:8081 falla: MockWebServer no se inició; revisar `@BeforeEach` en test base

## Anexos

### Estructura del proyecto de tests

```
bff-service/src/test/java/com/colegio/bffservice/  
├── AbstractMockWebServerTest.java  
│   └── Mock server dispatcher para /estudiantes, /asistencias, /evaluaciones  
├── AcademicoIntegrationTest.java  
│   └── 6 integration tests (BFF → Mock services)  
├── AcademicoE2ETest.java  
│   └── 7 E2E tests (flujos de negocio)  
└── BffServiceApplicationTests.java  
    └── 4 unit tests (controllers + mocks)
```

### Configuración de test (application-test.properties)

```
eureka.client.enabled=false  
eureka.client.fetch-registry=false  
eureka.client.register-with-eureka=false  
spring.cloud.discovery.enabled=false  
spring.cloud.enabled=false  
estudiante.service.url=http://localhost:8081  
asistencia.service.url=http://localhost:8081  
evaluacion.service.url=http://localhost:8081
```